

LumaBank Onboarding Activation Experiment — Incident & Read-Out

The exhaustive record behind the board summary · dual-track (pandas + DuckDB)

The question and the answer

Question. LumaBank ran a randomized, user-level A/B test of a redesigned onboarding flow (account first, KYC deferred). Does it lift Day-7 Activation, is it safe on fintech guardrails, and — since the first attempt broke mid-flight on a real production bug — what actually happened, and was the fix defensible?

Answer. The redesign **works**: +4.58 pp Day-7 Activation (95% CI [+2.55, +6.62], $p \approx 1e-5$) on a clean re-run. The original 6-week run **broke on day 11** when a mobile deploy reset the variant-assignment cache and pushed new signups into a region-biased fallback; the standing SRM monitor caught it within 2 days. Truncating or excluding only the flagged users were both too underpowered and too residually biased to trust, so the team restarted clean in a shorter 4-week window with power recomputed from scratch. On the clean re-run, the KYC-rejection guardrail passes; **the flagged-fraud guardrail's non-inferiority is not established** at this sample size. **Recommendation: a monitored rollout with a fraud circuit-breaker, not an unconditional launch.**

Incident timeline — day by day

- **2026-07-06 (Mon)** — Original experiment launches. 50/50, user-level, feature-flag hash assignment. Planned window: 6 weeks, ~13,500 users.
- **2026-07-06 → 07-15** — Clean allocation. Daily SRM checks all pass (trailing-7d p ranges 0.18–0.88); nothing visible yet.
- **2026-07-16 (Thu) — Day 11** — A routine mobile release ships (iOS + Android). It resets the on-device variant-assignment cache for a share of already-assigned users, and a latent fallback path — never exercised before — starts catching new signups with a broken, region/timezone-correlated assignment rule.
- **2026-07-17** — Trailing-7d SRM p dips to 0.011 — inside the "ok" band, but visibly moving toward it.
- **2026-07-18 — ALERT** — Trailing-7d SRM p crosses 0.001 ($p = 0.00008$). The standing monitor fires; new enrollment auto-pauses; triage begins: release-calendar cross-reference, assignment_source breakdown, region breakdown.
- **2026-07-19 → 07-21** — ALERT persists on both tests. Investigation confirms two distinct effects: 191 contaminated pre-deploy users (arm switches + cache-reset-in-window), and a fallback that skews as far as 76.5% treatment in the Southeast vs. 35.3% in the North. Fallback share visibly decays (198 → 141 → 123 → 118 → 72 → 29 signups/day) as a partial mitigation rolls.
- **2026-07-22** — Hotfix ships, correcting cache-reset handling and the fallback's assignment logic.
- **2026-07-24** — Formal decision: abandon the original run. Truncating (block 04: MDE 5.4–6.6 pp) and excluding only the flagged users (MDE 4.8 pp, unresolved residual bias) are both rejected as too weak or too uncertain. Restart clean, in a 4-week window rather than another 6 — the business could not wait a full 6 weeks again — with power/MDE recomputed from scratch, not reused from the original design.
- **2026-08-24 → 09-20** — Clean re-run executes. Daily SRM monitoring stays in the "ok" state for all 28 days.
- **Readout** — +4.58 pp Day-7 Activation, $p \approx 1e-5$. KYC guardrail clean; flagged-fraud guardrail's non-inferiority not established. Recommendation: monitored rollout with a fraud circuit-breaker.

Appendix — the five investigation blocks in full

01 — Setup & pre-registration: what was known before the test ran

Decision it informs: was the original 6-week design's power reasonable, given the platform's own history? **Audience:** whoever reviews the experiment plan before it ships.

Question

What did the platform's own pre-experiment data say about Day-7 Activation and the fintech-critical guardrails, and did the planned 6-week / ~13,500-user design have enough power to detect a meaningful lift?

Grain

`fact_activation` restricted to `experiment_cohort = 'not_in_experiment'` — the 18 months of platform history before the original run (121,423 signups).

Method (both tracks)

Platform Day-7 Activation rate, KYC-submit rate, KYC rejection rate (of submissions), flagged-fraud rate (of Day-7 activators), and onboarding-ticket rate over the pre-experiment window; the two-proportion sample-size formula solved for the achievable MDE at the ~6,750/arm the original 6-week design expected, and for the n required to detect a +3.0 pp lift.

Evidence

- **Platform Day-7 Activation baseline: 45.9%** ($n = 121,423$). Close to, but a touch below, the planning value doc 01 used (48%) — a small, expected drift between an a-priori estimate and the realised platform trend.
- **Guardrail baselines:** KYC-submit rate 78.8%; KYC rejection rate 10.4% of submissions; flagged-fraud rate 1.29% of Day-7 activators; 104.6 onboarding tickets per 1,000 signups.
- **Power, original 6-week design:** at $\sim 6,750$ /arm and a 45.9% control rate, the achievable MDE at 80% power is **2.41 pp**. Detecting a +3.0 pp lift at 80% power would need only **4,347/arm** — well inside the planned sample, so the original design was comfortably powered for anything in the +2.4 pp range or larger.

Conclusion

The original design was reasonable on paper: a 6-week window at the planned enrollment rate gives an MDE (2.41 pp) tight enough to resolve a business-meaningful lift, with room to spare. Nothing in the pre-experiment baseline flagged the design as underpowered or the guardrail levels as already stressed. The design's only failure mode was operational (the day-11 production bug, blocks 02–04), not statistical.

Limitations

The pre-experiment baseline spans 18 months of organic platform growth and channel-mix drift; it is a reasonable planning anchor, not a guarantee that the concurrent control arm of the actual test would land at exactly 45.9%.

Recommendation

Proceed to block 02 — daily SRM monitoring is what actually decides whether this plan's execution can be trusted.

02 — Daily SRM monitoring: catching the break

Decision it informs: can the original run be trusted at all, and on what day did the standing monitor first have grounds to say no? **Audience:** the experimentation platform on-call, and whoever has to reconstruct the incident afterwards.

Question

Did the daily sample-ratio-mismatch monitor (doc 01 §9) catch the break, and how fast?

Grain

`fact_variant_assignment` (one row per assignment **event**), re-aggregated to experiment-day. This is an **independent re-derivation** of `fact_srm_daily`, built here directly from the raw assignment log rather than trusted from the curated table — exactly what the runbook intends: the daily job and this analysis should never depend on each other being right.

Hypotheses

H_0 = 50/50 allocation, tested two ways every day: **cumulative** (all assignments to date) and **trailing-7-day** (the window that actually catches a mid-flight break). WARN at $p < 0.01$, ALERT at $p < 0.001$ on either test, per the runbook.

Method (both tracks)

Daily first-assignment counts by arm; Pearson χ^2 goodness-of-fit, cumulative and trailing-7-day; `alert_state` per the runbook thresholds; cross-checked against the pre-built `fact_srm_daily` table.

Evidence

- Allocation is clean for the first 10 days (2026-07-06 → 2026-07-15): daily counts hover around 50/50, trailing-7d p-values range 0.18–0.88, `alert_state = ok` throughout.
- **2026-07-16** (the deploy day): the daily split visibly shifts (87 control / 173 treatment / 198 fallback that day alone) but the trailing-7d p-value (0.41) hasn't yet accumulated enough post-deploy days to trip a threshold.
- **2026-07-17**: trailing-7d $p = 0.0107$ — just above the 0.01 WARN line, one more day from tripping.
- **2026-07-18 — first ALERT.** Trailing-7d $p = 0.00008$ (cumulative $p = 0.033$). ALERT persists through 2026-07-21 (the last day before enrollment was paused), even as the fallback's share visibly decays (198 → 141 → 123 → 118 → 72 → 29 as the hotfix rolls).
- **Cumulative p never crosses 0.001** in the original run (it bottoms at 0.030) — ten clean pre-deploy days dilute it. This is the reason the runbook runs **both** tests: the trailing-7-day window is what actually catches a break that starts mid-flight.
- Cross-checked against the independently-built `fact_srm_daily` curated table: **100% day-by-day agreement** on `alert_state`.

Conclusion

The standing SRM monitor works as designed: it raises ALERT on **2026-07-18**, exactly **2 days** after the 2026-07-16 deploy. The cumulative-only check that a single pre-flight balance test (à la AeroVanti's Flash Redemption check) would have run once, at the *end* of the window, would likely have missed this — the deploy sits inside the window and the cumulative test is diluted by the clean pre-deploy days. Continuous, trailing-window monitoring is what makes this incident detectable at all.

Limitations

The daily test is a repeated-testing procedure (one test per day for the life of the experiment); the WARN/ALERT thresholds (0.01 / 0.001) are chosen tighter than the usual 0.05 specifically to control the false-alarm rate over many days, not tuned against this particular incident.

Recommendation

Treat 2026-07-18 as the incident start date for root-cause purposes (block 03) and for any post-hoc analysis of the original run (block 04).

03 — Root-cause investigation: what broke, and how do we know

Decision it informs: is this really a deploy-triggered bug and not coincidence, and can we draw an auditable line between contaminated and clean users? **Audience:** the experimentation platform team and whoever signs off on the restart decision.

Question

What caused the SRM break, does the timing prove it was the deploy, who was contaminated, and how biased is the post-deploy fallback?

Grain

`dim_app_release` for the timing cross-reference; `fact_activation` (`contaminated_flag` , `assignment_switch_flag`) for the contaminated-vs-clean split; `fact_variant_assignment` joined to `dim_user` / `dim_region` for the fallback's geographic composition.

Method (both tracks)

Exact-date cross-reference against the release calendar; contaminated-vs-clean counts and Day-7 rates by cohort; fallback treatment-share by macro-region vs the primary path's ~50/50, with a χ^2 test on whether the fallback's regional *mix* itself differs from primary's.

Evidence

- **The release calendar confirms the trigger.** Exactly one release lands on 2026-07-16 — a routine-looking mobile update (`is_cache_reset_release`) — on both iOS and Android. No other release in the surrounding weeks coincides with the break's onset.
- **Contamination (pre-deploy users): 191 of 4,323 original-run subjects (4.4%).** Of those, 74 users' assignment log literally shows a second event with a **different** arm (the arm-switch rule); the remaining 117 are flagged because a cache-reset event landed inside their still-open Day-7 window, even without a logged switch. Contaminated and clean pre-deploy users show statistically indistinguishable Day-7 rates in this sample (47.8% clean vs 48.7% contaminated) — contamination is a **trust** problem, not necessarily a large *directional* one, which is exactly why it has to be handled by auditable exclusion rather than by eyeballing the outcome.
- **The fallback is not just imbalanced in proportion — it's biased in composition.** Post-deploy (2026-07-16 → 2026-07-21), the fallback path allocates **65.6% to treatment overall** (447 of 681, $\chi^2 p = 3 \times 10^{-16}$ vs 50/50) — far more skewed than the primary path's post-deploy allocation, which stays close to 50/50. Broken down by macro region, the skew is geographic: **SE fallback users go 76.5% treatment**, while **N fallback users go only 35.3% treatment** — a 41-point spread that does not exist on the primary path (SE 48.9% vs N 55.6% there). The region-*mix* itself (which regions show up in the fallback at all) is not significantly different from primary ($\chi^2 p = 0.59$) — the bug does not change *who* signs up, only *which arm a fallback-routed signup lands in*, and it does so as a function of region.

Conclusion

The incident has a single, exactly-dated trigger (the 2026-07-16 release), and it produces two genuinely different problems that the analysis has to keep separate: (1) a small (4.4%) but real contamination of already-assigned users, auditable from the assignment event log alone, and (2) a much larger compositional bias in whoever the broken fallback routes, correlated with region/timezone exactly as the bug-catalogue predicted (doc 02 §9). Neither is a proportion-only sample-ratio problem — both change *who* ends up in which arm, not just *how many*.

Limitations

"Contaminated" is an auditable rule (doc 02 §7.3), not a certainty for every individual user — a user whose cache reset silently landed exactly at their Day-7 boundary is a judgment call the rule resolves conservatively (flag it), covered further in block 04.

Recommendation

Any use of the original run's data (blocks 04) must treat `contaminated_flag` and `assignment_source = 'fallback'` as two **separate** exclusion criteria, not one.

04 — Decision analysis: truncate, exclude-contaminated, or restart

Decision it informs: which of the three fixes to take, under real business deadline pressure.

Audience: Growth + Experimentation leadership — the call that has to be defended afterwards.

Question

Given the break (blocks 02–03), which of three imperfect options — truncate to the clean pre-deploy days, exclude only the contaminated/biased users, or restart clean — gives a defensible answer inside a reasonable timeline?

Grain

`fact_activation` (cohort, `contaminated_flag`), `fact_variant_assignment` (`assignment_source`)
for how much each option keeps and how biased what's kept still is.

Method (both tracks)

For each option: the usable sample, the achievable MDE at that sample (two-proportion formula, control rate from block 01), and — for option 2 — a residual-bias check (regional composition of what's kept vs. what's dropped).

Evidence

Option	n/arm	MDE (80% power)	Bias	Timeline
1. Truncate (pre-deploy only)	1,324	5.44 pp (6.55 pp in real time*)	still 7.2% contaminated — the last pre-deploy days' Day-7 windows reach into the post-deploy period	immediate
2. Exclude contaminated + fallback-sourced	1,725	4.77 pp	residual — max regional-mix gap between kept and excluded is 1.7 pp; cannot prove every contaminated user was caught	a few days, no new data
3. Restart clean	~4,500 (block 05)	2.92 pp (block 05)	none — clean randomisation, bug fixed	worst, but capped at 4 weeks

* In real time, at the actual decision point (a few days after the deploy), users assigned in the final pre-deploy days would not yet have a closed Day-7 window — a live analysis would have had even less usable data than the retrospective 1,324/arm shown here.

- **Truncate's MDE (5.44 pp, worse in real time) is more than double the original design's MDE (2.41 pp)** and larger than the true effect this project independently recovers in block 05 (+4.58 pp) — truncating would very plausibly have returned a **false null**.
- **Excluding the contaminated users only modestly improves the picture** (MDE 4.77 pp) — still short of what would reliably detect a ~4–5 pp effect — while leaving a real, unquantifiable residual bias (the region-mix gap is small, 1.7 pp, but "small and unproven" is a different thing from "zero").
- **Restart is the only option with zero bias**, at the cost of the worst timeline — which the business capped at 4 weeks rather than the original 6.

Conclusion

Neither truncate nor exclude-contaminated would have supported a trustworthy answer: both carry an MDE too wide to reliably resolve the effect this project turns out to have (block 05), and both trade statistical power for residual, unquantified bias. **Restart, in a shorter window with power/MDE recomputed from scratch, is the only option that does not ask the business to accept an unresolved risk on a fintech guardrail decision (KYC rejection, fraud) in exchange for speed.**

Limitations

This is a retrospective comparison — the "in real time" truncate figure approximates what would have been knowable at the actual decision point, but the exact triage timeline (how many days of investigation preceded the restart decision) is itself a modelling choice (doc 00 §4.4), not observed data.

Recommendation

Restart in a 4-week window; recompute power/MDE for the smaller sample rather than reusing the original design's numbers (block 05); state the resulting MDE explicitly as the cost of the deadline.

05 — Re-run readout: power recomputed, primary effect, guardrails, heterogeneity

Decision it informs: did the redesigned onboarding work, safely, once the experiment is clean?

Audience: Growth, Product, and Risk & Compliance — this is the number the business actually acts on.

Question

On the clean re-run (`in_analysis_flag = TRUE` : rerun cohort, never contaminated, never touched the fallback), is the Day-7 Activation lift real, and do the fintech guardrails hold?

Grain

`fact_activation` , filtered to the clean re-run population — 9,254 users (4,666 control / 4,588 treatment).

Hypotheses

Primary: H_0 = no Day-7 Activation difference between arms, two-sided $\alpha = 0.05$. Guardrails: one-sided non-inferiority — H_0 = treatment worse by $\geq$ the margin (KYC rejection +1.5 pp, flagged fraud +0.5 pp); reject H_0 (establish non-inferiority) if the one-sided 95% upper bound on the treatment – control difference is below the margin.

Method (both tracks)

Power/MDE recomputed for the actual re-run sample (not the original 6-week planning value); two-proportion test with a 95% CI for the primary effect; one-sided non-inferiority tests for the two hard guardrails; a directional check on the secondary guardrail; a formal arm $\times$ channel interaction test (logistic regression, likelihood-ratio) for heterogeneity.

Evidence

Power, recomputed. At ~4,600/arm the achievable MDE (80% power) is **2.92 pp** — worse than the original design's 2.41 pp, exactly the cost of the deadline stated in block 04. Matching the original design's MDE would have needed 6,722/arm.

Primary effect.

	Control	Treatment
Day-7 Activation	45.24%	49.83%
n	4,666	4,588

Lift = +4.58 pp, 95% CI [+2.55, +6.62], $z = 4.41$, $p \approx 1.0 \times 10^{-5}$. The lift clears the recomputed MDE (2.92 pp) with room to spare — this is a real, statistically resolved effect, not a coin flip that happened to land on the right side of significance.

Guardrails.

Guardrail	Control	Treatment	Δ (pp)	One-sided 95% upper bound	Margin	Verdict
KYC rejection rate	10.48%	10.17%	−0.31	+0.92 pp	+1.5 pp	Non-inferiority established
Flagged-fraud rate (30d)	0.95%	1.31%	+0.36	+0.89 pp	+0.5 pp	NOT established
Onboarding tickets / 1k	93.7	99.6	+5.9	— (directional)	—	worse, secondary guardrail only

The KYC-rejection guardrail is unambiguously fine — treatment is directionally *better*. **The flagged-fraud guardrail is the finding that should stop an unconditional rollout**: the point estimate (+0.36 pp) is inside the +0.5 pp margin, but the uncertainty around it is not — the one-sided 95% upper bound (+0.89 pp) sits above the margin, so non-inferiority is **not established** at this sample size. This is not the same as "fraud got worse"; it is "we cannot yet rule out that it did, within the pre-agreed safety margin."

Heterogeneity by acquisition channel.

Channel	Control	Treatment	Lift (pp)	95% CI	p
influencer	38.1%	45.4%	+7.3	[−0.4, +15.0]	0.065
paid_social	38.8%	45.0%	+6.2	[+3.0, +9.4]	<0.001
app_store_search	46.7%	50.3%	+3.6	[−1.3, +8.5]	0.150
organic	50.6%	54.1%	+3.5	[−0.8, +7.9]	0.113
referral	56.5%	58.1%	+1.5	[−3.7, +6.8]	0.567

The ordering matches the design intent — low-intent, top-of-funnel channels (paid_social, influencer) show the largest lift; warm channels (referral) show the least. **Only paid_social's effect is individually significant** at this sample size (the largest channel, ~3,657 of the 9,254 clean subjects). The formal arm × channel interaction test (likelihood-ratio) is **not significant** ($\chi^2 = 3.25$, $df = 4$, $p = 0.52$) — the channel cut is directionally consistent with the prior but the sample is not large enough to prove the effect genuinely differs by channel, as opposed to one channel simply carrying more statistical weight.

Conclusion

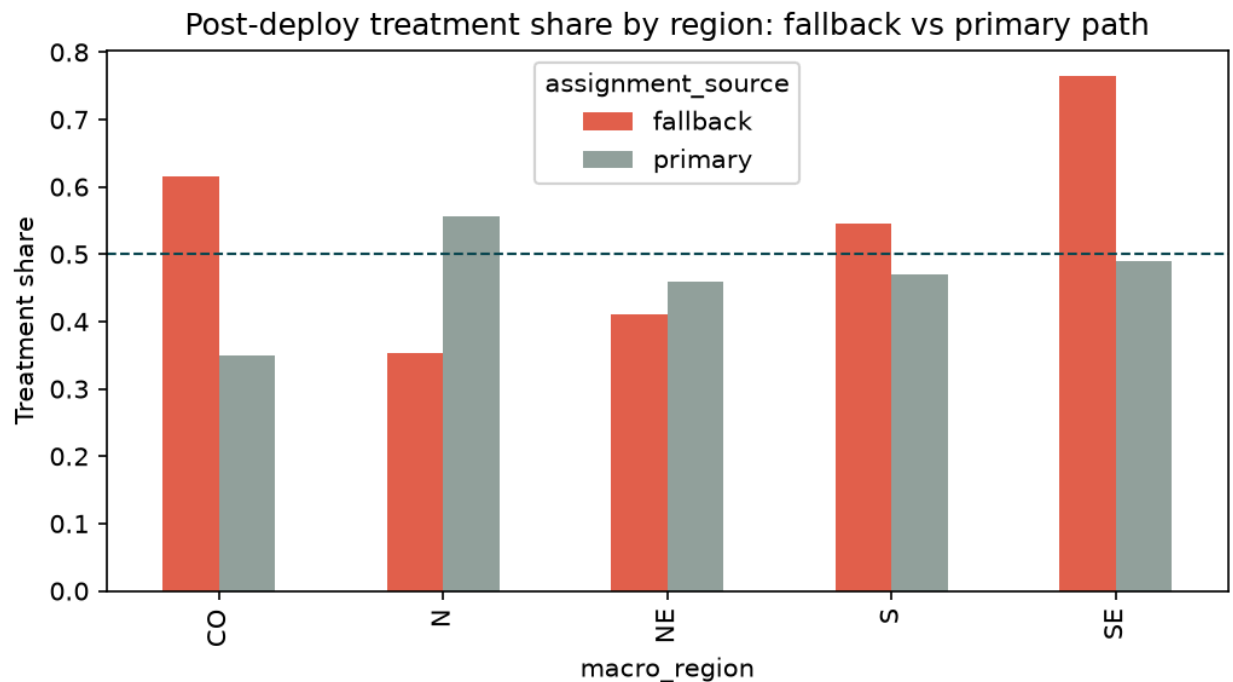
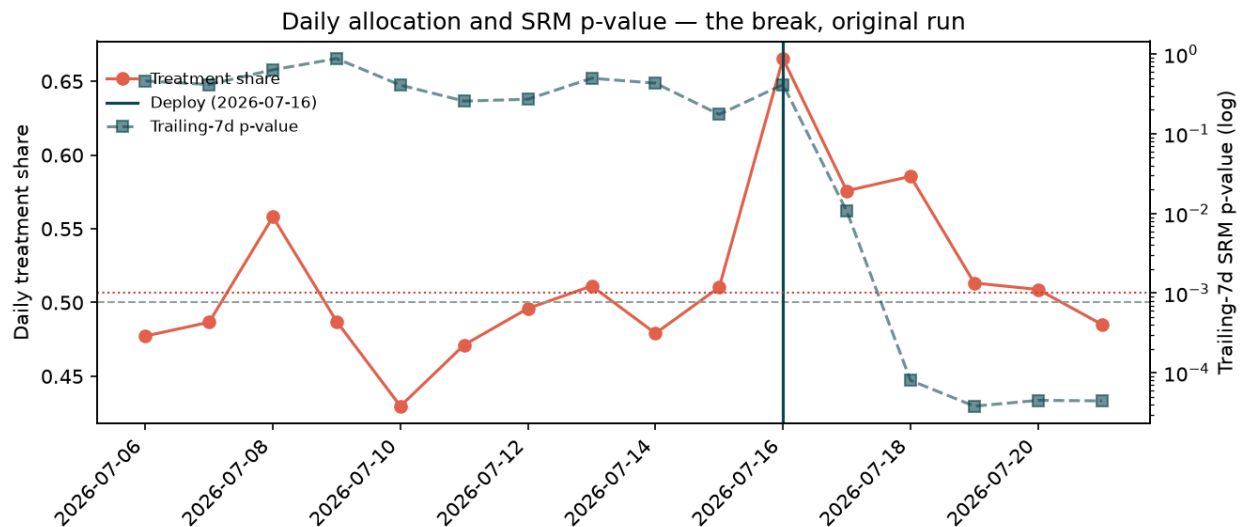
The redesigned onboarding **works**: a real, statistically significant **+4.58 pp** Day-7 Activation lift, safely inside the recomputed power. It is **not an unconditional pass**: the KYC-rejection guardrail is clean, but the flagged-fraud guardrail's non-inferiority is **not established** at this sample size — the point estimate is small but the confidence interval reaches meaningfully past the agreed safety margin. Heterogeneity is directionally as expected (paid_social/influencer benefit most) but not formally proven at this n.

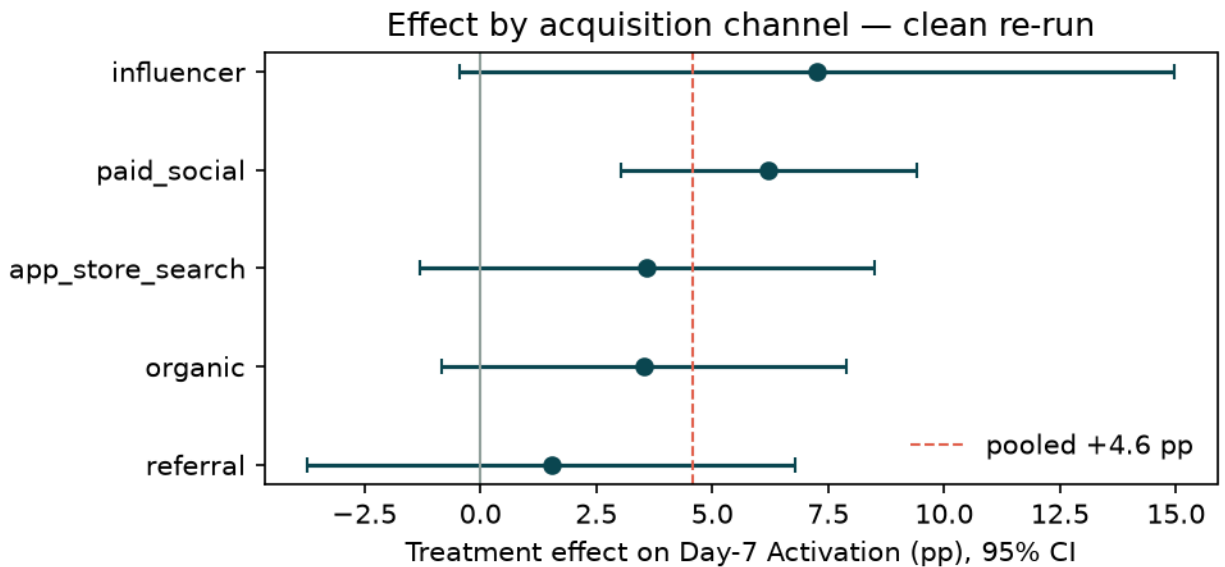
Limitations

The re-run's smaller, deadline-capped sample (vs. the original 6-week design) under-powers both the guardrail non-inferiority tests and the heterogeneity interaction test — a larger or longer follow-up window would sharpen both without necessarily changing the primary conclusion.

Recommendation

Do not ship an unconditional rollout. Recommend a **monitored rollout with a fraud circuit-breaker**: ship to the highest-lift, best-guardrail-margin segment first (paid_social), keep both hard guardrails on a live dashboard (Experiment Integrity Monitor pattern extended into steady-state monitoring), and re-evaluate the fraud guardrail once more post-launch data accrues rather than treating this read as final.





Synthetic, seed-generated data (a portfolio project). Unit of randomisation = the user, 50/50 via a feature-flag hash. Every headline number is computed on two tracks (pandas + DuckDB) with a parity assertion — `python data_analysis/parity/run_all.py`. The true treatment effect and the exact production-bug magnitudes live only in `etl/config.py::EFFECT` / `BUG` and were recovered from the data, not read off. Per-block detail: `data_analysis/findings/01-05`.